Laboratory Exercise 11

Topics

1. Sets
2. Dictionaries
3. Table searches

Discussion

A. Discuss the similarities and differences between:
   a. a list;
   b. a set;
   c. a dictionary.
B. Discuss whether a dictionary can have:
   a. two keys with the same value;
   b. two values with the same key.
C. If Python didn't provide the `set` container, but you needed it in a program, what kind of container could you use instead?

Exercises (in red = suggested ones to complete during the lab)

Part 1 – Complex Data Structures

***Goal**: For each of the following exercises, write a Python program that responds to the above requests. Complete at least one exercise during the lab session, and the remaining ones at home.*

11.1.1 Counting words. Write a program that counts the occurrences of each word in a text file, whose name is entered as input. Assume that the file contains only alphabetic or space characters (such as the `input.txt` file). The program must display all the words present, with the number of occurrences of each one next to it. [P8.2]

11.1.2 Most frequent words. Extend the program of exercise 11.1.1 so that it displays only the `5` most frequent words in the file, without considering articles, prepositions and conjunctions in the count. [P8.3]

[SUGGESTED] 11.1.3 Two strings. Write a program that asks the user to provide two strings, and then displays (avoiding character repetitions in the output):

I. the characters that appear in both strings;
II. characters that appear in one string but not in the other;
III. the (alphabetical) letters that do not appear in either string.

Tip: Use the `set()` function to transform a string into a set of characters. [P8.9]

[SUGGESTED] 11.1.4 Censor. Write a 'censor' program, which reads a file (`bad_words.txt`) containing a list of 'bad words' (such as `'sex'`, `'drugs'`, `'C++'` and so on), one per line, inserting them into a set. Then read another text file (`raw_text.txt`), which contains occurrences of some of the 'bad words' in question. The program has to rewrite the second file, generating a third (`censored_text.txt`) in which the content is the same, but with the difference that all

occurrences of words and sub-words corresponding to 'bad words' are replaced by a number of asterisks (`'*'`) equal to their length. [P8.14]

[SUGGESTED] 11.1.5 Sparse vectors. A sparse vector is a sequence of numbers, most of which are `0`. An efficient way to store a sparse vector is a dictionary, in which the keys are the positions in which non-zero values are present, and the values are the corresponding values in the sequence. For example, the sequence `0 0 0 0 0 4 0 0 0 2 9` would be represented by the dictionary `{5:4, 9:2, 10:9}`. Write a function, `sparse_array_sum(a, b)`, whose arguments are two such dictionaries, `a` and `b`. The function, without modifying the dictionaries passed as parameters, must return their sum vector as a sparse vector, where a value at position `i` is the sum of the values of `a` and `b` at position `i`. [P8.22]

11.1.6 Sieve of Eratosthenes. The Sieve of Eratosthenes is an iterative algorithm, also known in Ancient Greece, that calculates all prime numbers less than an integer n. With each iteration, the algorithm deletes all multiples of the minimum value present in the set, starting from the minimum value 2 and arriving at the minimum value $\sqrt{n}$. At the end of the last iteration, the numbers left in the set are the ones you requested. Define a function that implements the Sieve of Eratosthenes. First, define a set and insert all the integers from `2` to `n`. Then, eliminate from the set all multiples of `2`, except `2` (i.e., `4, 6, 8, 10, 12,` and so on, up to `n`). As a second step, eliminate all multiples of `3`, except `3` (i.e., `6, 9, 12, 15,` and so on, up to `n`). Continue in this way until you reach $\sqrt{n}$, deleting each time the multiples of the minimum value present in the set, until the numbers remaining in the set are those required. [P8.4]

Part 2 – Complex Operations

***Goal**: For each of the following exercises, write a Python program that responds to the above requests. Complete <u>at least one</u> exercise during the lab session, and the remaining ones at home.*

[SUGGESTED] 11.2.1 Per capita income. Write a program that reads data from the `rawdata_2004.txt` text file and puts them in a dictionary whose keys are names of countries and whose values are annual per capita incomes. Note that in the file, the fields are separated by a tab character `'\t'`. Then, the program must ask the user to provide country names as input, and display the corresponding values of annual income per capita. The program must terminate when the user enters the string `'quit'` as input. It is possible to read similar data, updated to 2021, in `.csv` format, from the `rawdata_2021.csv.` Try to solve the same exercise by working on this `.csv` file. [P8.17]

11.2.2 Genetic Code. In living cells, deoxyribonucleic acid (DNA) contains the fundamental information for the body to perform all the functions useful for life. To support the cell's functions, the instructions contained in DNA are first transcribed into ribonucleic acid (RNA) molecules. Then, these RNA molecules are translated into proteins. The translation from messenger RNA (mRNA) to proteins is based on the genetic code, i.e. the set of rules by which a sequence of nucleotides (`'A'`, `'C'`, `'U'` and `'G'`) is translated into a sequence of amino acids for protein synthesis. Each amino acid corresponds to one or more sequences of three nucleotides, which are called *codons*. The following table[1] shows the codons that encode the `20` ordinary amino acids in an mRNA.

1 Genetic code (Wikipedia): https://it.wikipedia.org/wiki/Codice_genetico

Aminoacid	Codons	Aminoacid	Codons
Ala	GCU, GCC, GCA, GCG	Leu	UUA, UUG, CUU, CUC, CUA, CUG
Arg	CGU, CGC, CGA, CGG, AGA, AGG	Lys	AAA, AAG
Asn	AAU, AAC	Met	AUG
Asp	GAU, GAC	Phe	UUU, UUC
Cys	UGU, UGC	Pro	CCU, CCC, CCA, CCG
Gln	CAA, CAG	Ser	UCU, UCC, UCA, UCG, AGU, AGC
Glu	GAA, GAG	Thr	ACU, ACC, ACA, ACG
Gly	GGU, GGC, GGA, GGG	Trp	UGG
His	CAU, CAC	Tyr	UAU, UAC
Ile	AUU, AUC, AUA	Val	GUU, GUC, GUA, GUG

The following codons indicate the start and end instructions for the translation process. Note that the start codon also encodes Methionine (`'Met'`).

Instruction	Codons
start	AUG, GUG
stop	UAG, UGA, UAA

Write a program that processes an mRNA sequence inserted as input by the user, and displays the amino acid sequence generated by it. To simulate the translation process, use a `genetic_code` dictionary, which, by reading the information reported in the table contained in the `codice_genetico.csv` file, stores the codons and amino acids encoded by them, choosing in an appropriate way which category to use as keys and which as values. Then, scroll through the sequence entered by the user until you find a match with one of the `'start'` codons. Finally, scroll through the next part of the sequence, storing the result of the translation of each codon into a `protein` list. The translation must stop when a `'stop'` codon is encountered. For example, starting from the sequence of nucleotides GUAUGCACGUGACUUUCCUCAUGAGCUGAU (the `'start'` and `'stop'` codons are underlined), the program should display: MetHisValThrPheLeuMetSerstop. If the scroll reaches the end of the sequence without encountering both the `'start'` and `'stop'` codons, then the program must return an error message.

11.2.3 Labyrinth. Write a program that reads a text file (`maze.txt`) containing an image of a maze, such as the following, in which the asterisks (`'*'`) represent walls and the spaces (`' '`) represent walkable corridors.

```
* *******
*   * * *
* ***** *
* * *   *
* * *** *
*   *   *
***** * *
*     * *
******* *
```

Create a `corridors` dictionary whose keys are tuples `(row, column)` of locations corresponding to a corridor and whose values are sets of locations that also correspond to a corridor, and are adjacent to the location specified by the respective key. In the example above, the key corresponding to the tuple `(1, 1)`, highlighted in blue, has `{(1, 2), (0, 1), (2, 1)}` as adjacent positions. Then display the resulting dictionary. [P8.20]

11.2.4 Ariadne's thread. Ariadne's thread owes its fame to the myth of Minos and the Labyrinth. In fact, it was used by Theseus to find the exit from Minos' labyrinth, tracing the direction towards the exit at each crossroads. Extend the program of exercise 11.2.3 so that, starting from any point, it finds a way out of the maze, for example by using the following algorithm:

> Based on the `corridors` dictionary, build a new `paths` dictionary whose keys are tuples `(row, column)` of positions corresponding to a corridor and whose values are initially all equal to the string `'?`. Then, iterate through the `paths` dictionary keys and, for each key that represents positions at the edges of the maze, and therefore corresponds to a possible exit, replace the string `'?'` with a string indicating the direction of the exit path: `'N'` (for 'North'), `'E'` (for 'East'), `'S'` (for 'South') or `'W'` (for 'West').
> Once this substitution is made, iterate again through the `paths` dictionary keys whose corresponding value is still equal to `'?`. For each of these locations, check to see if they have at least one adjacent location whose value in `paths` is different from `'?'`tag. If yes, replace the string `'?'` with a string indicating the direction in which the identified adjacent location is found. To find adjacent locations, use the `corridors` dictionary.
>
> *Example*: the key corresponding to the tuple `(1, 1)` has `{(1, 2), (0, 1), (2, 1)}` as adjacent locations. If the key `(0, 1)` has been assigned the value `'N'`, in the `paths` dictionary, and if in the current iteration the value assigned to the key `(1, 1)` is still `'?'`, we can replace the `'?'` with `'N'` since (0,1) is "North" with respect to (1, 1) and it has a known path to an exit. If no substitutions can be made during one of the key and value iterate iterations, finish running the program.

Finally, print the maze again, but displaying in each corridor position the direction that leads to the closest exit, as identified by the algorithm above. For example:

```
*N*******
*NWW*?*S*
*N*****S*
*N*S*EES*
*N*S***S*
*NWW*EES*
*****N*S*
*EEEEN*S*
*******S*
```

[P8.21]